
Adaptable Bayesian Neural Networks (ABNNs)

CS772 Project Report

Anmoldeep Singh Dhillon (220163)
dhillon22@iitk.ac.in

Naman Kumar Jaiswal (220687)
namankj22@iitk.ac.in

Pallav Goyal (220747)
pallavg22@iitk.ac.in

Prakhar Upadhyay (220788)
prakhar22@iitk.ac.in

Abstract

Despite remarkable advances in the predictive capabilities of deep neural networks (DNNs), efficient and robust uncertainty quantification remains essential to deploy them in safety-critical applications, yet traditional Bayesian neural networks (BNNs) are notoriously unstable and unscalable on large pre-trained models. Our approach, Adaptable Bayesian Neural Networks (ABNN) [4], is a post-hoc transfer-learning strategy that retrofits any pre-trained deterministic DNN with minimal computational overhead, achieves near-state-of-the-art accuracy alongside efficient epistemic uncertainty estimation.

1 Problem Description

Modern deep neural networks (DNNs) $f_\theta(\cdot)$ achieve state-of-the-art predictive performance across a wide range of tasks [1]. However, they produce only deterministic point estimates $\hat{y} = f_\theta(x)$ (θ being the Neural Network parameters) and offer no measure of epistemic uncertainty — a critical limitation in safety-sensitive applications such as autonomous driving, healthcare diagnostics, and anomaly detection. In high-stakes settings, even confidently wrong predictions can lead to catastrophic failures, highlighting the need for reliable uncertainty quantification (UQ) [16].

Bayesian neural networks (BNNs) aim to address this by modeling a posterior distribution $p(\theta \mid \mathcal{D})$ over parameters, enabling the estimation of predictive uncertainty. However, exact inference is intractable, and approximate methods like variational inference suffer from scalability issues, instability, and significant computational overhead when applied to large models with millions of parameters. This makes traditional BNNs impractical for real-world deployment.

Hence, there is a pressing need for scalable models with meaningful uncertainty estimates, which require minimal fine-tuning, preserve the original predictive performance, and introduce negligible computational cost - enabling uncertainty-aware decision making without the burdens of full Bayesian training.

We aim to fulfill the following objectives with our course project:

1. **Develop a Scalable Uncertainty Estimation Model:** Design a lightweight strategy to equip pre-trained deterministic DNNs with uncertainty estimation capabilities, with minimal overhead.
2. **Benchmark Predictive and Uncertainty Performance:** Evaluate the accuracy and uncertainty quality of the proposed method compared to standard widely-used deterministic models.
3. **Establish Theoretical Foundations:** Provide formal justification for the method's ability to approximate the posterior (Uncertainty) in the vicinity of the pretrained model's optimum.
4. **Understand Model Behavior and Limits:** Analyze the generalization, convergence behavior, and limitations of the method across different architectures and tasks.

2 Literature Review

Bayesian neural networks (BNNs) extend deep learning by placing a prior $p(\theta)$ over network parameters θ and computing the posterior $p(\theta | \mathcal{D})$ via Bayes’ rule. Early work applied Markov Chain Monte Carlo (MCMC) methods to sample exact posteriors in small networks (Neal, 1995) [13] and introduced the Laplace approximation for analytical tractability. Variational inference (VI) then enabled scalable posterior approximations by fitting simple distributions $q(\theta)$, as in Graves (2011) [7] and Bayes by Backprop (Blundell et al., 2015) [3], though these often underestimate uncertainty under the mean-field assumption.

To make uncertainty estimation practical in large networks, several modern methods trade exactness for efficiency. Monte Carlo Dropout (Gal & Ghahramani, 2016) [5] interprets dropout at inference as approximate Bayesian averaging. Deep Ensembles (Lakshminarayanan et al., 2017) [10] train multiple models and average predictions, yielding strong calibration at the cost of linear compute scaling. SWAG (Maddox et al., 2019) [11] fits a Gaussian to SGD trajectories for fast posterior sampling, while Laplace-based methods (Kristiadi et al., 2020) [9] approximate local curvature around the MAP for uncertainty with minimal retraining.

More recent approaches exploit existing network components or post-hoc modules to inject uncertainty without retraining all weights. Building on these ideas, our Adaptable Bayesian Neural Network (ABNN) strategy (Franchi et al., 2023) [4] retrofits pretrained DNNs by replacing normalization layers with Bayesian Normalization Layers—attaching Gaussian perturbations to the scale parameters—and fine-tuning only these adaptations, achieving near-ensemble uncertainty at single-model cost.

3 Probabilistic Viewpoint: Bayesian Neural Networks (BNNs)

3.1 Probabilistic Perspective

Let θ denote the parameters (weights & biases) of a neural network. We shall model these parameters as random variables and reason about their posterior distribution given data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$.

The likelihood of the data under parameters θ is:

$$p(\mathcal{D} | \theta) = \prod_{i=1}^N p(y_i | x_i, \theta)$$

We place a prior over the parameters, often a zero-mean isotropic Gaussian, $p(\theta) = \mathcal{N}(\theta; 0, \sigma^2 I)$ acting as an L2 regularization term, penalizing large weights and favoring smoother functions.

The posterior distribution is then given by Bayes’ Rule:

$$p(\theta | \mathcal{D}) = \frac{p(\mathcal{D} | \theta)p(\theta)}{p(\mathcal{D})}$$

Maximum A Posteriori (MAP) estimation seeks the mode of the posterior equivalent to minimizing the negative log posterior:

$$\theta^* = \arg \max_{\theta} \log p(\mathcal{D} | \theta) + \log p(\theta) = \arg \min_{\theta} - \sum_{i=1}^N \log p(y_i | x_i, \theta) + \lambda \|\theta\|^2 \quad (1)$$

3.2 Bayesian Neural Networks (BNNs)

BNNs generalize DNNs by treating each weight w_{ij} as a random variable (even normalization layer biases and weights are random variables)[15]. Rather than a single point estimate θ^* , we learn an approximate posterior $q(\theta) \approx p(\theta | \mathcal{D})$, having the same support Ω and, at inference, draw samples $\theta^{(k)} \sim q(\theta)$ to compute the predictive distribution

$$p(y^* | x^*, \mathcal{D}) = \int_{\theta \in \Omega} p(y^* | f_{\theta}(x^*)) q(\theta) d\theta \approx \frac{1}{K} \sum_{k=1}^K p(y^* | f_{\theta^{(k)}}(x^*)), \quad \theta^{(k)} \sim q(\theta)$$

3.3 Approximation Methods for BNNs

Several approximate inference methods are used to train BNNs:

1. Variational Inference (VI)

- Approximate the posterior $p(\theta \mid \mathcal{D})$ with a tractable variational distribution $q(\theta)$ [14].
- Minimize the Kullback-Leibler divergence:

$$\text{KL}(q(\theta) \parallel p(\theta \mid \mathcal{D}))$$

- This is equivalent to maximizing the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{VI}} = \mathbb{E}_{q(\theta)} [\log p(\mathcal{D} \mid \theta)] - \text{KL}(q(\theta) \parallel p(\theta))$$

- Optimization is done using the reparameterization trick: sample $\theta = \mu + \sigma \cdot \epsilon$, where $\epsilon \sim \mathcal{N}(0, 1)$.

2. Monte Carlo (MC) Dropout

- Train a DNN with dropout enabled during both training and inference [5].
- At test time, perform multiple stochastic forward passes with dropout active.
- For multiple samples of weight masks θ_t , predictive distribution is approximated as:

$$p(y \mid x, \mathcal{D}) \approx \frac{1}{T} \sum_{t=1}^T p(y \mid x, \theta_t), \quad \theta_t \sim q(\theta)$$

3. Deep Ensembles

- Train multiple DNNs independently with different random initializations and data shuffling [10].
- Predictive mean and variance are computed from the ensemble:

$$\mathbb{E}[y] = \frac{1}{K} \sum_{k=1}^K f_{\theta_k}(x)$$

3.4 Challenges of BNNs

Despite their theoretical appeal, BNNs face significant practical limitations:

- **Intractability:** Computing the true posterior is computationally intractable.
- **Training difficulty:** Approximations like VI and MCMC are hard to tune and slow to converge.
- **Unstable convergence:** High variance gradients and complex loss surfaces hinder reliable training.
- **Scalability:** BNNs are resource-intensive and scale poorly to large models or datasets.
- **Number of Parameters:** Now each weight parameter is represented by some distribution parameters, which make the number of parameters multiple folds.

As a result, simpler alternatives like MC Dropout and Deep Ensembles are often preferred in practice.

4 Our Methodology

4.1 Normalization Layers

Modern DNNs commonly include normalization layers (e.g. batch or layer normalization) to stabilize learning by centering and scaling intermediate activations [8]. Concretely, batch normalization computes per-batch mean μ and variance σ^2 , and produces outputs via an affine rescaling:

$$y_i = \gamma \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

where γ, β are learnable scale and shift parameters. This operation ensures that each normalized feature has zero mean and unit variance before the affine transform.

4.2 Bayesian Normalization Layers (BNLs)

ABNN injects uncertainty into a deterministic network by perturbing the scale parameters of each normalization layer with Gaussian noise. In a Bayesian Normalization Layer (BNL), the usual affine scale γ_i is replaced by a stochastic scale $\gamma_i(1 + \zeta_i)$, where ζ_i is a zero-mean Gaussian random variable. Formally, for each channel or feature i the BNL outputs

$$\tilde{y}_i = \gamma_i(1 + \zeta_i) \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i, \quad \zeta_i \sim \mathcal{N}(0, \sigma_\zeta^2), \quad (2)$$

with μ, σ^2 the usual batch-mean and variance. This simple modification (adding noise to the scale) turns the deterministic layer into a Bayesian one, allowing the network to model predictive uncertainty.

4.3 ABNN Loss function: Perturbed Regularized Cross Entropy Loss

The ABNN fine-tuning minimizes a perturbed MAP objective over the Bayesian Normalization Layer parameters $\phi = \{\gamma_j, \beta_j\}$, initialized at their pretrained values ϕ_0 . Concretely, the loss combines a weighted negative log-likelihood and an L_2 prior:

$$\mathcal{L}_{\text{ABNN}}(\phi) = - \sum_{i=1}^N \eta_i \log p(y_i | f_{\omega^*}(x_i; \phi)) + \frac{1}{2\sigma^2} \|\phi - \phi_0\|^2. \quad (3)$$

Here ω^* are the frozen pretrained weights, and each η_i is a small random perturbation (e.g. sampled from $\{1 - \delta, 1, 1 + \delta\}$) applied to the NLL term. These perturbations encourage different instances of the ABNN to settle in diverse local minima, boosting ensemble diversity, while the Gaussian prior (L_2 regularization) keeps the adapted parameters close to their original values.

4.4 Training

We convert a pre-trained DNN into an ABNN by replacing each normalization layer with a BNL (initializing γ, β from the pretrained model). All original network weights are then frozen, and only the BNL parameters (γ, β) are trained on the ABNN loss function. In practice, we typically train multiple ABNN instances (with different initial noise seeds) to capture multiple modes of the posterior. Key steps in training are:

- **Replace norm layers:** For each batch/instance norm in the pretrained model, substitute a BNL initialized with the pretrained γ, β .
- **Freeze base weights:** Lock all original weights (convolutional and linear layers), so that only the BNL parameters (γ, β) are updated.
- **Fine-tune with perturbed loss:** Optimize the modified network using perturbed cross-entropy, which adds small random weights to each class in the loss to prevent collapse.

4.4.1 Training Pseudocode

Algorithm 1 ABNN Training Procedure

Require: Pre-trained DNN with normalization layers, learning rate η , epochs T

- 1: Replace each normalization layer with a BNL, initializing its γ, β from the pretrained model
 - 2: Freeze all original network weights; only BNL parameters (γ, β) are trainable
 - 3: **for** $t = 1$ to T **do**
 - 4: **for** each minibatch (x, y) **do**
 - 5: Sample fresh Gaussian noise ζ for each BNL ($\zeta_i \sim \mathcal{N}(0, 1)$)
 - 6: Compute network outputs $p = f(x; \gamma, \beta, \zeta)$ via forward pass
 - 7: Compute modified cross-entropy loss $\ell(p, y)$ with random class-weight perturbations
 - 8: Update only γ, β using $\nabla_{\gamma, \beta} \ell$ via back-propagation
 - 9: **end for**
 - 10: **end for**
-

4.5 Inference

At test time, each BNL injects different noise per forward pass, so we perform multiple stochastic passes to estimate uncertainty. Concretely, given a new input x , we repeat:

- Sample new Gaussian noise in each BNL layer.
- Compute the network output (e.g. softmax probabilities).
- Average the outputs over M passes to form the predictive distribution.

This Monte Carlo averaging yields the predictive mean and uncertainty of the ABNN. In practice we take

$$\hat{p} = \frac{1}{M} \sum_{m=1}^M f(x; \gamma, \beta, \zeta^{(m)}),$$

with independent $\zeta^{(m)}$ samples each pass. The inference pseudocode is:

Algorithm 2 ABNN Inference for uncertainty prediction

Require: Test input x , trained ABNN model, number of passes M

- 1: $\hat{p} \leftarrow 0$ (initialize sum of probabilities)
 - 2: **for** $m = 1$ to M **do**
 - 3: Sample fresh Gaussian noise ζ for each BNL ($\zeta_i \sim \mathcal{N}(0, 1)$)
 - 4: Compute softmax output $p^{(m)} = f(x; \gamma, \beta, \zeta)$
 - 5: $\hat{p} \leftarrow \hat{p} + p^{(m)}$
 - 6: **end for**
 - 7: $\hat{p} \leftarrow \hat{p}/M$ (average the outputs)
 - 8: **return** $\hat{p}$ (predictive probability)
-

5 Attempted Novelty

In standard cross-entropy loss, each class is typically treated with equal importance, represented as $\alpha(y_i) = 1$. In contrast, we have incorporated a *random prior* by introducing random weight perturbations η_i for certain classes, modifying the weighting to $\alpha(y_i) = 1 + \eta_i$. This strategy encourages different ABNNs to specialize in different class subsets. The results have been mentioned in Table 3. As a result, the overall training objective becomes:

$$\mathcal{L}(\omega) = \mathcal{L}_{\text{MAP}}(\omega) + \mathcal{E}(\omega) \quad (4)$$

where

$$\mathcal{E}(\omega) = - \sum_{(x_i, y_i) \in \mathcal{D}} \eta_i \log P(y_i | \mathbf{x}_i, \omega). \quad (5)$$

If `random_prior` is enabled, the class-wise weights η_i are drawn from a Dirichlet distribution scaled by the number of classes K :

$$\boldsymbol{\eta} \sim \text{Dirichlet}(\mathbf{K}), \quad (6)$$

where $\boldsymbol{\eta} = [\eta_1, \eta_2, \dots, \eta_K]$ represents the random prior weights for each class.

We also tried variations of ABCNN (Adaptive Bayesian CNN) by different combinations of deterministic normalisation layers and Bayesian normalization layers, which we call the **Mixed ABNN**. All our models are made in PyTorch and trained on Nvidia RTX 3060M. The results are mentioned in Table 3.

6 Experiments and Results

6.1 Codebase

Our codebase has been pushed to GitHub. The file named `Analysis.ipynb` contains the experiments done. Furthermore we have taken inspiration from an **unofficial** publically available repository of the same paper[12]

6.2 Datasets

We used the **CIFAR10** dataset. The data was pre-processed and transformed using standard techniques of normalization and random horizontal flipping. In each case, we measure metrics of ABNNs relative to the performance of DNN models and measure their uncertainty quantification abilities.

6.3 Experiments

Following the original paper, we trained a CNN on CIFAR10 dataset and transformed it into an ABNN. The CNN’s normalization layers, **viz.** BatchNorm and LayerNorm were transformed into BNL i.e. Bayesian Normalization Layer of the form (2) that inherits from `torch.nn.Module`. The models were then retro-trained for fine tuning.

We also tried multiple loss functions and compared results across all kinds of training routines. Further we have also tried random prior to generate perturbations in loss functions as mentioned above.

- `torch.nn.CrossEntropyLoss()`: Standard Cross-Entropy loss
- CustomMAPLoss: Custom defined loss function i.e. the Negative Log Likelihood regularized by a **Normal** log prior as shown in equation (1). This regularized loss is added with a perturbation term i.e. Negative Log Likelihood weighted with a user-defined vector `eta` which is for our purposes `torch.ones(num_classes)`
- ABNNLoss: This is just CustomMAPLoss regularized by `log_softmax` probabilities weighted by a user-defined vector `eta` as show in (3)

6.4 Results

We have compared the results of CNN with its ABNN form and **Mixed ABNN** form, and ResNet-50 and its ABNN form in Table 1. All models have been trained using `torch.nn.CrossEntropyLoss`

For both architectures (CNN and ResNet-50) in Table 1, **ECE** has dropped by a significant amount as was expected since Bayesian Inference decreases over-confident predictions. This is due to the fact that **DNNs** produce high-confidence incorrect predictions whereas **ABNNs** produce more uncertain decisions through diverse predictions.

Moreover in Table 1, for mixed layer ABNN, it can be seen that **FPR95** (False Positive Rate at 95% recall) decreases as compared to fully **Bayesianized** model, which means it can be chosen which layers to convert to **Bayesian Normalization Layer** to give better results. In Table 2, highest performance among CNNs was achieved by **Mixed ABNN**.

It can be seen in Table 3 that a Bayesian Model with **Random Prior** is hard to train. We can further experiment with other forms of distribution to sample the perturbation weights from, like a **Multivariate Gaussian Prior**. **ECE** was expected to fall significantly when using random priors with **ABNN**, since they allow specialization of group of ABNNs over classification.

Furthermore, it is necessary to retro-train the ABNN i.e. retrain the model after reloading pre-trained weights and changing architecture to a Bayesian model. Simply superimposing previously trained DNN on an ABNN architecture yields poor performance as evident in Table 4.

We didn’t train the ResNet-50 architecture for other cases in **Table 2** to **Table 4** due to large computational overhead and long training time.

Table 1: Comparison of Single Model and ABNN on CNN and ResNet-50 for CIFAR10 using Cross Entropy Loss

	Method	Acc $\uparrow$	NLL $\downarrow$	ECE $\downarrow$	AUPR $\uparrow$	AUC $\uparrow$	FPR95 $\downarrow$
CIFAR-10	CNN						
	Single Model	78.4	0.867	0.13	86.5	97.4	12.6
	ABNN	79.1	0.928	0.03	75.5	94.8	23.5
	Mixed ABNN	80.6	0.806	0.03	80.5	96.0	18.39
	ResNet-50						
	Single Model	90.1	0.611	3.1	95.2	91.9	23.6
	ABNN	89.5	0.712	0.04	97.0	94.1	11.4

Table 2: Comparison of Single Model and ABNN on CNN using Other kinds of Loss Funtions

Method		Acc $\uparrow$	NLL $\downarrow$	ECE $\downarrow$	AUPR $\uparrow$	AUC $\uparrow$	FPR95 $\downarrow$
CIFAR-10	CNN						
	Single Model (CustomMapLoss)	79.7	0.862	0.123	87.6	97.6	11.1
	ABNN (CustomMapLoss)	77.3	0.914	0.03	76.4	95.0	22.9
	ABNN (ABNNLoss)	77.8	0.881	0.01	77.2	95.2	22.9
	Mixed ABNN (ABNNLoss)	80.84	0.818	0.03	80.5	96.0	19.0

Table 3: Comparison of Single Model and ABNN on CNN and ResNet-50 for CIFAR10 using Random Priors

Method		Acc $\uparrow$	NLL $\downarrow$	ECE $\downarrow$	AUPR $\uparrow$	AUC $\uparrow$	FPR95 $\downarrow$
CIFAR-10	CNN						
	Single Model (Random Prior)	79.3	0.864	0.12	87.3	97.6	11.9
	ABNN (Random Prior)	68.24	0.928	0.04	75.7	94.7	24.3

Table 4: Performance of non retro-trained ABNN based on CNN

Method		Acc $\uparrow$	NLL $\downarrow$	ECE $\downarrow$	AUPR $\uparrow$	AUC $\uparrow$	FPR95 $\downarrow$
CIFAR-10	CNN						
	ABNN (Untrained)	40.0	3.95	0.44	38.4	80.2	68.5

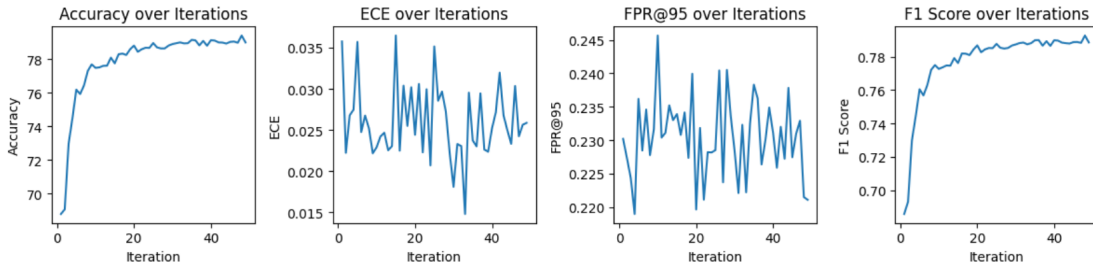


Figure 1: Illustration of model improvement with number of parallel ABNNs

7 Potential Applications and Future Works

7.1 Potential Applications

ABNNs enable pretrained perception models to report calibrated uncertainty with minimal overhead, making them highly suited for safety-critical systems such as autonomous driving, where novel obstacles or adverse weather conditions can be flagged via elevated epistemic uncertainty rather than overconfident mispredictions [2]. ABNNs also find interesting use cases in medical imaging, and also to provide uncertainty-aware outputs in NLP or multimodal tasks such as translation and image captioning [17]. Finally, in active learning scenarios, ABNNs’ per-sample uncertainty scores guide the selection of the most informative unlabeled samples for annotation, thereby reducing labeling costs and accelerating model improvement and capability[6].

7.2 Future Work

Extending ABNNs to architectures without conventional normalization (e.g. certain graph neural networks or custom recurrent networks) would broaden their applicability, suggesting exploration of analogous Bayesian adaptation modules in those settings. Hybridizing ABNNs with weight-space methods such as SWAG

(Stochastic Weight Averaging–Gaussian) could capture both activation-level and covariance-level uncertainty, yielding richer predictive ensembles while still leveraging pretrained backbones [11].

8 Learnings

Through this project, we gained valuable insights and practical experience in the field of Machine Learning research. One of our key learnings was how to conduct a literature review — a first for us. This process helped us understand the intuition behind the approaches proposed by the paper’s authors. We observed how they arrived at uncertainty estimation while also aiming to reduce computational cost and execution time.

We explored how existing methods could be adapted and combined with concepts learned during the course to achieve improvements in model accuracy and efficiency. Additionally, through hands-on experimentation and coding, we developed a comprehensive understanding of the paper’s methodologies.

9 Acknowledgement

This project was completed collaboratively in a team, with all members contributing equally to the research, implementation, analysis, and documentation. Anmoldeep and Prakhar collaborated to the codebase and experimentation, while Naman and Pallav worked towards ideation and research. Documentation was done together by the team.

We hereby declare that the work presented in this report is our own, carried out as part of the CS772 Course Project requirements. The content does not contain any form of plagiarism, and proper credit has been given wherever due, including references to existing literature, code, and ideas. This is a fresh project and we have not earned any credit elsewhere for this work.

References

- [1] Alexander Amini, Ava Soleimany, et al. Introduction to deep learning. <https://introtodeeplearning.com/>, 2024. Massachusetts Institute of Technology (MIT) 6.S191.
- [2] Fabio Arnez, Huascar Espinoza, Ansgar Radermacher, and François Terrier. A comparison of uncertainty estimation approaches in deep learning components for autonomous vehicle applications. *arXiv preprint arXiv:2006.15172*, 2020.
- [3] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *International conference on machine learning*, pages 1613–1622. PMLR, 2015.
- [4] Gianni Franchi, Olivier Laurent, Maxence Leguéry, Andrei Bursuc, Andrea Pilzer, and Angela Yao. Make me a bnn: A simple strategy for estimating bayesian uncertainty from pre-trained models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12194–12204, 2024.
- [5] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR.
- [6] Yarin Gal, Riashat Islam, and Zoubin Ghahramani. Deep bayesian active learning with image data. In *International Conference on Machine Learning (ICML)*, pages 1183–1192, 2017.
- [7] Alex Graves. Practical variational inference for neural networks. *Advances in neural information processing systems*, 24, 2011.
- [8] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- [9] Agustinus Kristiadi, Matthias Hein, and Philipp Hennig. Learnable uncertainty under laplace approximations. In *Uncertainty in Artificial Intelligence*, pages 344–353. PMLR, 2021.
- [10] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.
- [11] Wesley J Maddox, Pavel Izmailov, Timur Garipov, Dmitry P Vetrov, and Andrew Gordon Wilson. A simple baseline for bayesian uncertainty in deep learning. *Advances in neural information processing systems*, 32, 2019.
- [12] Abtin Mogharabin and Abdualloh Damash. Makeme-bnn: Unofficial implementation of "make me a bnn". <https://github.com/abtinmU/MakeMe-BNN>, 2024.
- [13] Radford M Neal. *Bayesian learning for neural networks*, volume 118. Springer Science & Business Media, 2012.
- [14] Princeton University. Cos597c: Theoretical machine learning, 2011.
- [15] Stanford University. Cs221: Artificial intelligence: Principles and techniques, 2019.
- [16] Amit Yadav. Uncertainty quantification in deep learning. <https://medium.com/data-scientists-diary/uncertainty-quantification-in-deep-learning-4d1a4daaa5af>, October 2024.
- [17] Ke Zou, Zhihao Chen, Xuedong Yuan, Xiaojing Shen, Meng Wang, and Huazhu Fu. A review of uncertainty estimation and its application in medical imaging. *Meta-Radiology*, 1(1):100003, 2023.